

Relatório de Acompanhamento - Sprint 1: Projeto FoodGuard

1. Definição de Escopo

- **Situação Problema:** A necessidade de identificar componentes alergênicos em produtos alimentícios para prevenir reações adversas nos usuários.
- **Solução Escolhida:** Uma plataforma web, vinculada à disciplina de Extensão III, para interpretação de rótulos via código de barras ou comandos de texto, cruzando os dados nutricionais com a anamnese do usuário.
- **Plataforma Alvo:** O escopo do projeto é estritamente voltado para a plataforma Web. Não haverá desenvolvimento de aplicativo mobile. A responsividade da interface não é o foco principal no momento e só será implementada em etapas futuras se houver tempo hábil.
- **MVP:** Optou-se por não definir um MVP fechado durante esta sprint, priorizando apenas o levantamento das necessidades e a criação de uma fundação tecnológica básica.

2. Definição de Arquitetura

A arquitetura foi dividida entre o servidor (Python/Django) e o cliente (React), focando apenas na instalação e configuração inicial das ferramentas, sem implementações robustas de lógica de negócio ou segurança nesta sprint.

- **Backend (API):**
 - Desenvolvido em Python utilizando o framework Django (v4.2.16) e Django REST Framework para a construção das APIs.
 - O servidor rodará de forma assíncrona utilizando Uvicorn (uvicorn[standard], uvicorn-worker).
 - A autenticação de sessões e rotas foi preparada utilizando pacotes como django-allauth[mfa] e PyJWT.
 - Foram adicionadas bibliotecas de suporte como Pillow (processamento de imagens base), hiredis (comunicação com Redis) e reportlab.
 - Ferramentas de customização do painel administrativo (django-unfold, django-admin-autocomplete-filter) e de formulários (django-crispy-forms, crispy-tailwind) foram incorporadas.
 - Nota sobre Segurança: O pacote argon2-cffi foi adicionado à base do projeto, mas a decisão técnica sobre como a criptografia das credenciais e dados médicos será efetivamente configurada foi postergada para sprints futuras. A documentação das rotas será feita com drf-spectacular.
- **Frontend (Interface Web):**
 - Construído em React (v19) com roteamento gerenciado pelo react-router-dom.
 - O consumo da API Django será feito utilizando axios, e o gerenciamento de estados assíncronos e cache das requisições utilizará o @tanstack/react-query.
 - A interface de usuário (UI) utilizará Tailwind CSS (via tailwind-merge e clsx) em conjunto com componentes acessíveis baseados no Radix UI e Shadcn.

- A iconografia será providenciada pelos pacotes lucide-react e react-icons, enquanto a tipografia padrão utiliza a fonte @fontsource-variable/geist.
- Utilitários adicionais como date-fns e react-day-picker foram adicionados para a manipulação futura de datas (como a data de nascimento no cadastro).

3. Definição de Requisitos

- Foi estruturado o documento formal "FoodGuard Requisitos - Extensão 3 (1).pdf" contendo o mapeamento das funcionalidades.
- Foram levantados 17 Requisitos Funcionais, abrangendo fluxos desde o cadastro da anamnese até a edição do prompt de chat e exclusão de conversas.
- O documento especifica 4 Requisitos de Qualidade e 14 Regras de Negócio fundamentais, como a proteção de dados na comunicação com o LLM e a obrigatoriedade da anamnese para liberar o chat.

4. Definição de Prototipação de Aplicação

- O desenho das interfaces visuais começou a ser rascunhado para refletir os requisitos de login, cadastro, anamnese e a tela principal de interação com o chatbot.
- A página "Home", que conteria as seções informativas "Quem somos" e "Como funciona", foi removida do escopo de prototipação desta sprint para que o esforço ficasse retido nos fluxos básicos de autenticação e chat.

5. Início da Implementação de API para Leitura de Código de Barras

- Foram iniciados estudos e testes preliminares, de forma não robusta, com a biblioteca zbar-wasm para extração do código de barras da imagem.
- O planejamento engloba o uso futuro do modelo Yolov8n integrado com ONNX Runtime-web para processar a visão computacional localmente.

6. Início da Implementação do Frontend

- O repositório frontend foi inicializado com as dependências listadas na arquitetura.
- O foco esteve apenas em estabelecer o esqueleto do projeto (configuração do Tailwind e arquivos de roteamento do React Router), sem a construção das páginas em si.

7. Organização Entre Membros da Equipe

A distribuição de responsabilidades entre os membros da equipe (Aristóteles Feitosa, Gabriel Pinheiro, Paulo Matheus, Pedro Otávio e Victor Manoel) foi organizada da seguinte maneira:

- **Gabriel, Paulo e Pedro:** Ficaram responsáveis pela elaboração da documentação e pelo levantamento e estruturação dos requisitos do sistema.

- **Victor:** Assumiu a responsabilidade pela prototipação das interfaces. A validação do artefato gerado contou com a participação ativa de todos os integrantes da equipe.
- **Aristóteles:** Ficou responsável por definir e estruturar a arquitetura técnica do projeto no backend e sua conexão com o frontend.

8. Dificuldades Encontradas

- **Clareza na Documentação:** Dificuldade na construção e redação dos requisitos do sistema, exigindo um esforço adicional para escrevê-los da maneira mais clara possível e sem ambiguidades.
- **Coleta de Dados de Saúde:** Definição complexa sobre o nível de detalhes e quais informações específicas seriam coletadas no guia de saúde do usuário (anamnese) para garantir a eficácia do modelo sem comprometer a usabilidade ou a privacidade.
- **Questões éticas:** Definição do limiar onde até que ponto podemos desenvolver um chat que auxilie os usuários sem comprometer ou tentar invalidar a importância de um diagnóstico ou acompanhamento profissional.
- **Definição da Plataforma Alvo:** Discussões prolongadas para a tomada de decisão sobre a viabilidade de construir uma versão mobile do aplicativo ou manter o foco exclusivo no ambiente web.
- **Integração de Inteligência Artificial:** Dificuldade na decisão arquitetural sobre a utilização ou não de RAG (Retrieval-Augmented Generation) para embasar as respostas do chatbot com o contexto do usuário.